QGENTIC AI

An Automated ML Competition Stack

THE PROBLEM

Kaggle gold medals require 200+ hours of work against thousands of competitors. Most time (~80%) goes to repetitive execution: debugging crashes, monitoring training, iterating models, evaluating submissions.

200+hrs	~20hrs	90%
Manual effort per gold medal	With Qgentic AI (human strategic input)	Effort reduction via LLM agents

Insight: LLMs can now handle the structured execution loop. Keep the human as strategic director.

RELATED WORK & POSITIONING

	Code	LLM	Comp	Multi	X-Lrn
Auto-sklearn	✗	✗	✗	✗	✗
AutoGluon	✗	✗	✗	✗	✗
Copilot/Cursor	✓	✓	✗	✗	✗
AIDE/ AutoKaggle	✓	✓	✓	✗	✗
Qgentic AI	✓	✓	✓	✓	✓

Key differentiators:

- AutoML: Fixed search spaces, no code generation or reasoning
- Copilot/Cursor: Code completion, not end-to-end pipelines
- AIDE/AutoKaggle: Closest, but single-agent, no cross-model learning
- **Qgentic AI: Multi-agent research + parallel execution + shared learning**

SYSTEM ARCHITECTURE

5 Coordinated LLM Agents

Starter — Task Classification ▶ **Researcher** — Deep Research Loop ▶ **Model Recommender** — Strategy Selection ▶ **Developer** — Code Gen & Iteration (xN) ▶ **Results** — Aggregation & Ranking

ORCHESTRATOR — Phase Sequencing | Parallel Execution | GPU/CPU Isolation | Checkpoint/Rollback

StarterAgent

Single LLM call with Google Search. Classifies task type (tabular, NLP, CV, time series) and generates summary for downstream agents.

`Output: starter_suggestions.json`

ResearcherAgent

Up to 512-step agentic tool-calling loop. Tools: execute_python (EDA + image ingestion), read_research_paper (ArXiv), scrape_web_page (Firecrawl).

`Output: plan.md`

ModelRecommender

3-stage: 16 candidates → paper analysis → 8 models. Per-model: preprocessing, loss, hyperparams, inference. MUST_HAVE vs NICE_TO_HAVE.

`Output: model_recommendations.json`

DeveloperAgent (parallel, one per model)

Generate train.py ▶ Guardrails ▶ Execute ▶ Monitor Logs 120s ▶ Score ▶ SOTA Search

◀ iterate

Tools: `execute_code`, `monitor_logs`, `search_red_flags`, `search_sota_suggestions`, `investigate_library`

Key Technical Innovations

Cross-Model Learning

Model A ↔ Shared Pool (thread-safe) ↔ Model B

✓ Cosine annealing (+0.02)

✓ MI feature selection (+0.015)

✗ Label smoothing (-0.01)

✗ Heavy augmentation (-0.005)

Safety & Guardrails

AST Static Analysis

logging.basicConfig order, data partition patterns

LLM Leakage Review

Gemini 3.1 Pro detects train/test contamination

LLM Code Safety

eval/exec, injection, credential leakage (Gemini 2.5 Flash)

Pipeline: evaluate_guardrails() runs before every code execution. Blocked code → feedback for regeneration.

Infrastructure & Resource Management

MIG
Single GPU partitioned

Multi-GPU
Separate GPUs per worker

CPU-Only
CPU affinity pinning

5 days
Baseline budget

12 hrs
Script timeout

120s
Log monitor poll

Isolated conda envs per model | ThreadPoolExecutor | Queue-based GPU/CPU pools | SQLite checkpoints + rollback | HITL via INSTRUCTIONS.md

RESULTS

CSIRO Biomass Competition

0.63772
Leaderboard Score

32 / 3,802
Final Ranking

Top 1%
Silver Medal

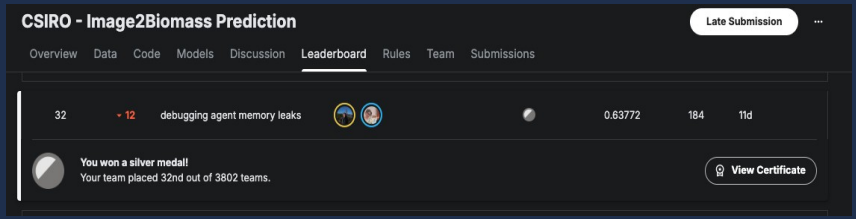
Human Effort Reduction

Traditional: 200+ hours

Qgentic AI: ~20 hrs

90% REDUCTION

Kaggle Leaderboard



LESSONS LEARNED

LLM Log Monitoring Works

Catches overfitting, NaN losses, stalled training that humans miss during off-hours

Cross-Model Learning Prevents Redundancy

Shared pool eliminates the "try the same bad idea 8 times" failure mode

Structured Outputs Are Essential

Pydantic schemas for LLM responses make multi-agent communication reliable

Robust Retry Logic Is Critical

Exponential backoff (1s'60s) + 5-min polling on sustained 503 keeps system running

ROADMAP & CONCLUSION

What's Next

Phase 1
Observability
Weave → Langfuse
Better tracing & cost tracking

Phase 2
Enhanced Guardrails
Lint integration
Improved leakage detection

Phase 3
Cross-Worker V2
Richer shared state:
embeddings, feature importances

Phase 4
Legal Retrieval
Competition rules
compliance (scaffolded)

★ **Ultimate Goal: Kaggle Competitions Grandmaster (2 more gold medals needed)**

Let the agents compete while you strategize

KEY TAKEAWAYS

Multi-agent decomposition beats monolithic agents	Specialized agents with distinct tools outperform one agent trying to do everything
Cross-model learning creates emergent optimization	Shared suggestion pools enable collective intelligence across parallel baselines
90% effort reduction is achievable today	Current LLM capabilities are sufficient for structured ML execution loops

Open Source: github.com/bogoconic1/Qgentic-AI